

链表

此处是用数组模拟链表

不使用 `struct node { int val
node * next }`

new node | 这样太慢了

① 单链表

常见用途：链接表存图、树

② 双链表

常见用途：优化

① 单链表



head 头结点.

head $\rightarrow \phi$ 空链表

head $\xrightarrow{\text{val}} 0 \xrightarrow{\text{next}} 0 \rightarrow 0 \rightarrow \phi$

数组模拟

$e[N]$ 表示某个点的 val

$ne[N]$ 表示某个点的 next 指针

关联：下标

head $\rightarrow \overset{3}{0} \rightarrow \overset{5}{0} \rightarrow \overset{7}{0} \rightarrow \overset{9}{0} \rightarrow \phi$, 空链表下标用 -1

0号结点的 val $e[0] = 3$

0号结点的 next 指针 $ne[0] = 1$ 指向 1 号点

$e[1] = 5$ $ne[1] = 2$

$e[2] = 7$ $ne[2] = 3$

$e[3] = 9$ $ne[3] = -1$

实际使用

int head, e[N] ne[N] idx

head: 头指针 e[i]: 结点 i 的值 ne[i]: 结点 i 的 next 指针

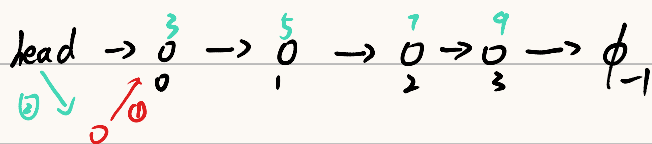
idx: 当前已经用到了哪个点 head 在这里不是结点, 而是一个指针

初始化:

init: head = -1; idx = 0

插入:

① 头插 (尾插的话可以再记录一个 tail 指针)



① 新结点指向原来的头

$e[idx] = x;$

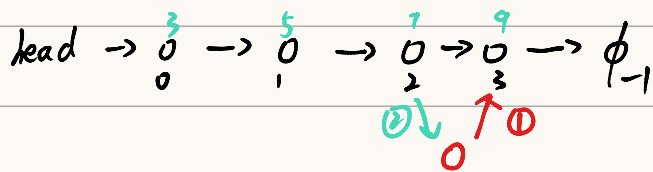
② 更新 head 的指向

$ne[idx] = head;$

$head = idx;$

$idx++;$

② 一般的插入, 插到下标是 k 的结点之后



$e[idx] = x$

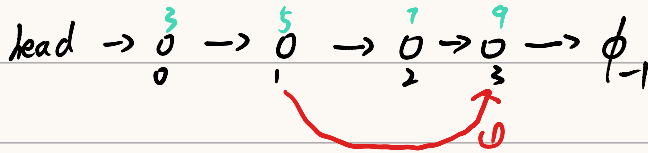
$ne[idx] = ne[k]$

$ne[k] = idx$

$idx++$

删除

① 将下标为 k 的结点的下一个结点删掉



$$ne[k] = ne[ne[k]]$$

这里实际并没有删掉, 只是这个位置我们不再用

省去了 free 的时间 以空间换时间

826. 单链表

题目 提交记录 讨论 题解 视频讲解

实现一个单链表，链表初始为空，支持三种操作：

1. 向链表头插入一个数；
2. 删除第 k 个插入的数后面的一个数；
3. 在第 k 个插入的数后插入一个数。

现在要对该链表进行 M 次操作，进行完所有操作后，从头到尾输出整个链表。

注意：题目中第 k 个插入的数并不是指当前链表的第 k 个数。例如操作过程中一共插入了 n 个数，则按照插入的时间顺序，这 n 个数依次为：第 1 个插入的数，第 2 个插入的数，... 第 n 个插入的数。

输入格式

第一行包含整数 M ，表示操作次数。

接下来 M 行，每行包含一个操作命令，操作命令可能为以下几种：

1. **H x**，表示向链表头插入一个数 x 。
2. **D k**，表示删除第 k 个插入的数后面的数（当 k 为 0 时，表示删除头结点）。
3. **I k x**，表示在第 k 个插入的数后面插入一个数 x （此操作中 k 均大于 0）。

输出格式

共一行，将整个链表从头到尾输出。

数据范围

$1 \leq M \leq 100000$

所有操作保证合法。

输入样例：

```
10
H 9
I 1 1
D 1
D 0
H 6
I 3 6
I 4 5
I 4 5
I 3 4
D 6
```

输出样例：

```
6 4 6 5
```

难度：

简单

时/空限制：

1s / 64MB

总通过数：

133672

总尝试数：

204245

来源：

模板题

AcWing

算法标签

```

1 #include <stdio.h>
2
3 const int N = 1e5 + 10;
4 int n;
5 int head, e[N], ne[N], idx;
6 // head头指针, idx最后一个元素的下标 + 10
7 // e存储val, ne存储next指针
8 void init() { // 初始化
9     head = -1;
10    idx = 0;
11    return ;
12 }
13
14 void add_to_head(int x) { // 头插
15     e[idx] = x;
16     ne[idx] = head;
17     head = idx;
18     idx ++;
19     return ;
20 }
21
22 void add(int k, int x) { // 中间插入
23     e[idx] = x;
24     ne[idx] = ne[k];
25     ne[k] = idx;
26     idx ++;
27     return ;
28 }
29
30 void del(int k) { // 删除 注意, 第一个节点删除后head也会变化, 需要特判
31     if(k == -1) head = ne[head];
32     else ne[k] = ne[ne[k]];
33     return ;
34 }
35
36 int main() {
37     init();
38     scanf("%d", &n);
39     char c;
40     int x, k;
41     for(int i = 0; i < n; i++) {
42         scanf("%c", &c); // 这里需要加个空格跳过换行符等内容!!
43         if(c == 'H') { // %c不会自动跳过换行符!!!
44             scanf("%d", &x);
45             add_to_head(x);
46         }
47         else if(c == 'D') {
48             scanf("%d", &k);
49             del(k - 1); // 这里我们k从0开始, 所以要 - 1
50         }
51         else {
52             scanf("%d%d", &k, &x);
53             add(k - 1, x);
54         }
55     }
56
57     for(int i = head; i != -1; i = ne[i]) printf("%d ", e[i]);
58     return 0;

```


② 双链表



这里我们不再额外定义 head tail

选取 0 号位为 head, 1 号位为 tail

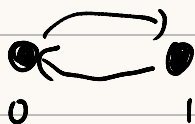
实现: 记录左指针 记录右指针

$e[N]$, $L[N]$, $R[N]$, idx

初始化:

$L[1] = 0$, $R[0] = 1$,

$idx = 2$

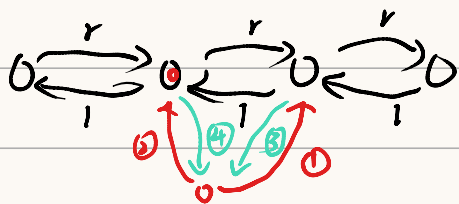


左端点右侧是右端点, 右端点左侧是左端点

idx 从 2 开始

插入

① 在 k 右侧插入



① ② $R[idx] = R[k]$

$L[idx] = k$

③ ④ $L[R[k]] = idx$

$R[k] = idx$

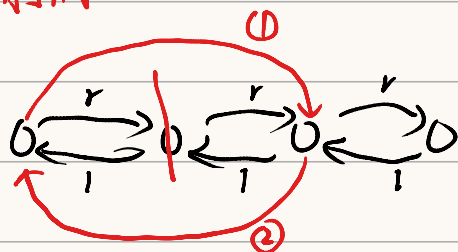
③ ④ 两步顺序不能错

② 左侧插入 同理

$R[idx] = k$ $L[idx] = L[k]$

$R[L[k]] = idx$ $L[k] = idx$

删除



$$\textcircled{1}: R[L[k]] = R[k]$$

$$\textcircled{2}: L[R[k]] = L[k]$$

邻接表

$head[1] \rightarrow \phi$

$head[2] \rightarrow \phi$

$\vdots$

$head[n] \rightarrow 0 \rightarrow 0 \rightarrow 0 \rightarrow \phi$

827. 双链表

题目

提交记录

讨论

题解

视频讲解

实现一个双链表，双链表初始为空，支持 5 种操作：

1. 在最左侧插入一个数；
2. 在最右侧插入一个数；
3. 将第 k 个插入的数删除；
4. 在第 k 个插入的数左侧插入一个数；
5. 在第 k 个插入的数右侧插入一个数

现在要对该链表进行 M 次操作，进行完所有操作后，从左到右输出整个链表。

注意：题目中第 k 个插入的数并不是指当前链表的第 k 个数。例如操作过程中一共插入了 n 个数，则按照插入的时间顺序，这 n 个数依次为：第 1 个插入的数，第 2 个插入的数，... 第 n 个插入的数。

输入格式

第一行包含整数 M ，表示操作次数。

接下来 M 行，每行包含一个操作命令，操作命令可能为以下几种：

1. `L x`，表示在链表的最左端插入数 x 。
2. `R x`，表示在链表的最右端插入数 x 。
3. `D k`，表示将第 k 个插入的数删除。
4. `IL k x`，表示在第 k 个插入的数左侧插入一个数。
5. `IR k x`，表示在第 k 个插入的数右侧插入一个数。

输出格式

共一行，将整个链表从左到右输出。

数据范围

$1 \leq M \leq 100000$

所有操作保证合法。

输入样例：

```
10
R 7
D 1
L 3
IL 2 10
D 3
IL 2 7
L 8
R 9
IL 4 7
IR 2 2
```

输出样例：

```
8 7 7 3 2 9
```

难度：

简单

时空限制：

1s / 64MB

总通过数：

85176

总尝试数：

126543

来源：

模板题

AcWing

算法标签

```

1  #include <stdio.h>
2  #include <string.h>
3
4  const int N = 1e5 + 10;
5  int n;
6  int e[N], L[N], R[N], idx;
7  // e存储val, L存储左指针, R存储右指针, idx表示当前位置
8
9  // 初始化
10 void init() {
11     // 0 表示左端点, 1表示右端点
12     // 0点右侧是1点, 1点左侧是0点
13     R[0] = 1; L[1] = 0;
14     idx = 2; // 0 1已经使用过了
15     // 此时R[0]表示最左端的结点
16     // L[1]表示最右端的结点
17 }
18
19
20 // 插入
21 void add_to_right(int k, int x) {
22     e[idx] = x;
23     R[idx] = R[k]; L[idx] = k; // 新结点调整指向
24     L[R[k]] = idx; // k右结点的左连到新节点
25     R[k] = idx; // k结点右连到新节点, 这两步不能颠倒
26     idx++;
27     return;
28 }
29
30 void add_to_left(int k, int x) {
31     e[idx] = x;
32     R[idx] = k; L[idx] = L[k];
33     R[L[k]] = idx; // k结点的左节点的右侧连到新节点
34     L[k] = idx; // k结点左侧连接到新节点
35     idx++;
36     return;
37 }
38
39 void add_L(int x) {
40     e[idx] = x;
41     R[idx] = R[0];
42     L[idx] = 0; // 先把新结点的线接好
43     L[R[0]] = idx; // 原最左端结点的左侧连到新节点
44     R[0] = idx; // 更新虚拟右结点
45     idx++;
46     return;
47 }
48
49 void add_R(int x) {
50     e[idx] = x;
51     L[idx] = L[1];
52     R[idx] = 1;
53     R[L[1]] = idx; // 原最右侧结点的有段连接到新节点
54     L[1] = idx; // 更新虚拟左节点
55     idx++;
56     return;
57 }
58
59 void del(int k) {
60     R[L[k]] = R[k]; // 左边结点的右侧连接到待删除的k的右侧
61     L[R[k]] = L[k]; // 右边结点的左侧连接到待删除的k的左侧
62     return;
63 }
64
65
66 int main() {
67     init();
68     scanf("%d", &n);
69     int k, x;
70     char in[10];
71     for(int i = 0; i < n; i++) {
72         scanf("%s", in);
73         if(strcmp(in, "R") == 0) {
74             // 字符串的比较要用strcmp, 不能直接 == !
75             scanf("%d", &x);
76             add_R(x);
77         } else if(strcmp(in, "L") == 0) {
78             scanf("%d", &x);
79             add_L(x);
80         } else if(strcmp(in, "IR") == 0) {
81             scanf("%d%d", &k, &x);
82             add_to_right(k + 1, x);
83             // 这里要注意, 之前我们自己定义了一个head, 我们的idx从0开始
84             // 所以我们使用k的时候是 - 1
85             // 但是现在我们已经占用了0 1, idx从2开始, 而我们题目中的k是
86             // 从1开始的, 因此我们要 + 1而不是 - 1 !
87         } else if(strcmp(in, "IL") == 0) {
88             scanf("%d%d", &k, &x);
89             add_to_left(k + 1, x);
90         } else {
91             scanf("%d", &k);
92             del(k + 1);
93         }
94     }
95     // 这里i不能从0开始, 0是虚拟左结点, R[0]才是左侧第一个有意义的结点
96     for(int i = R[0]; i != 1; i = R[i]) printf("%d ", e[i]);
97     return 0;
98 }

```